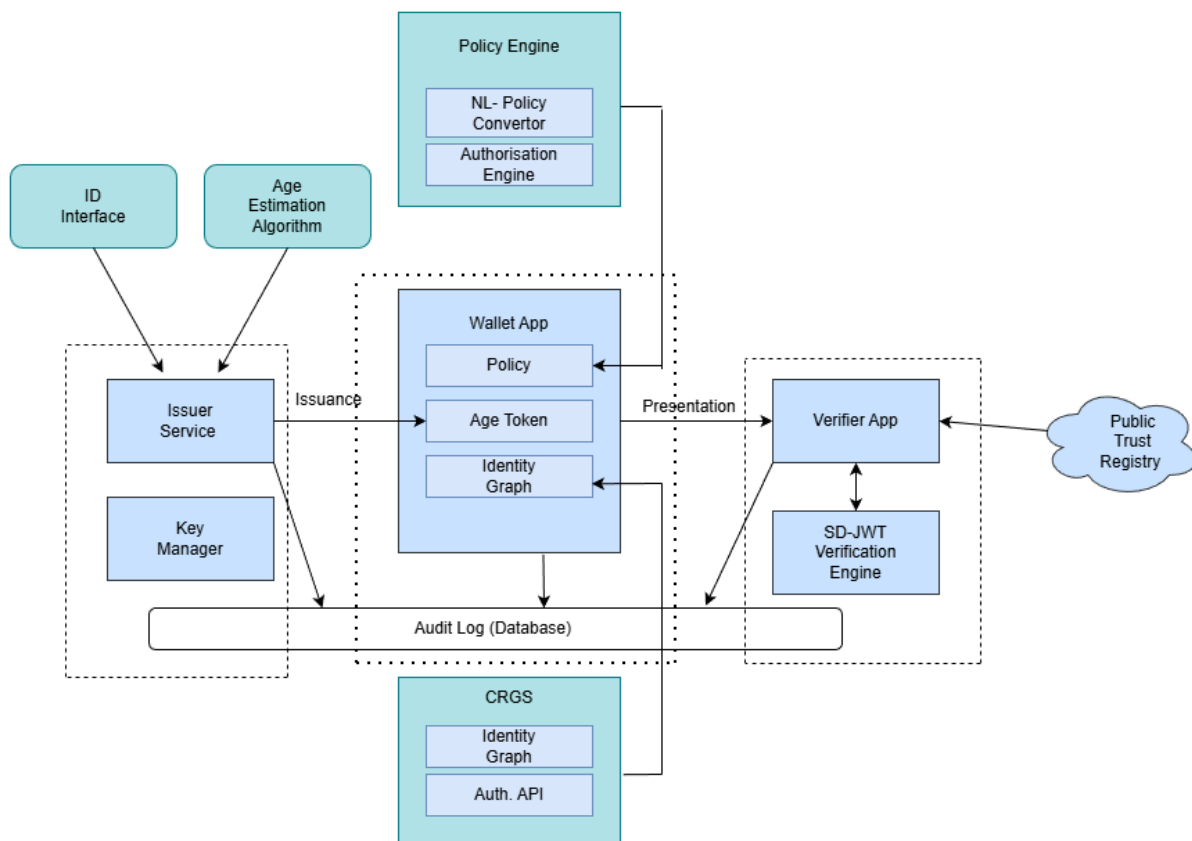
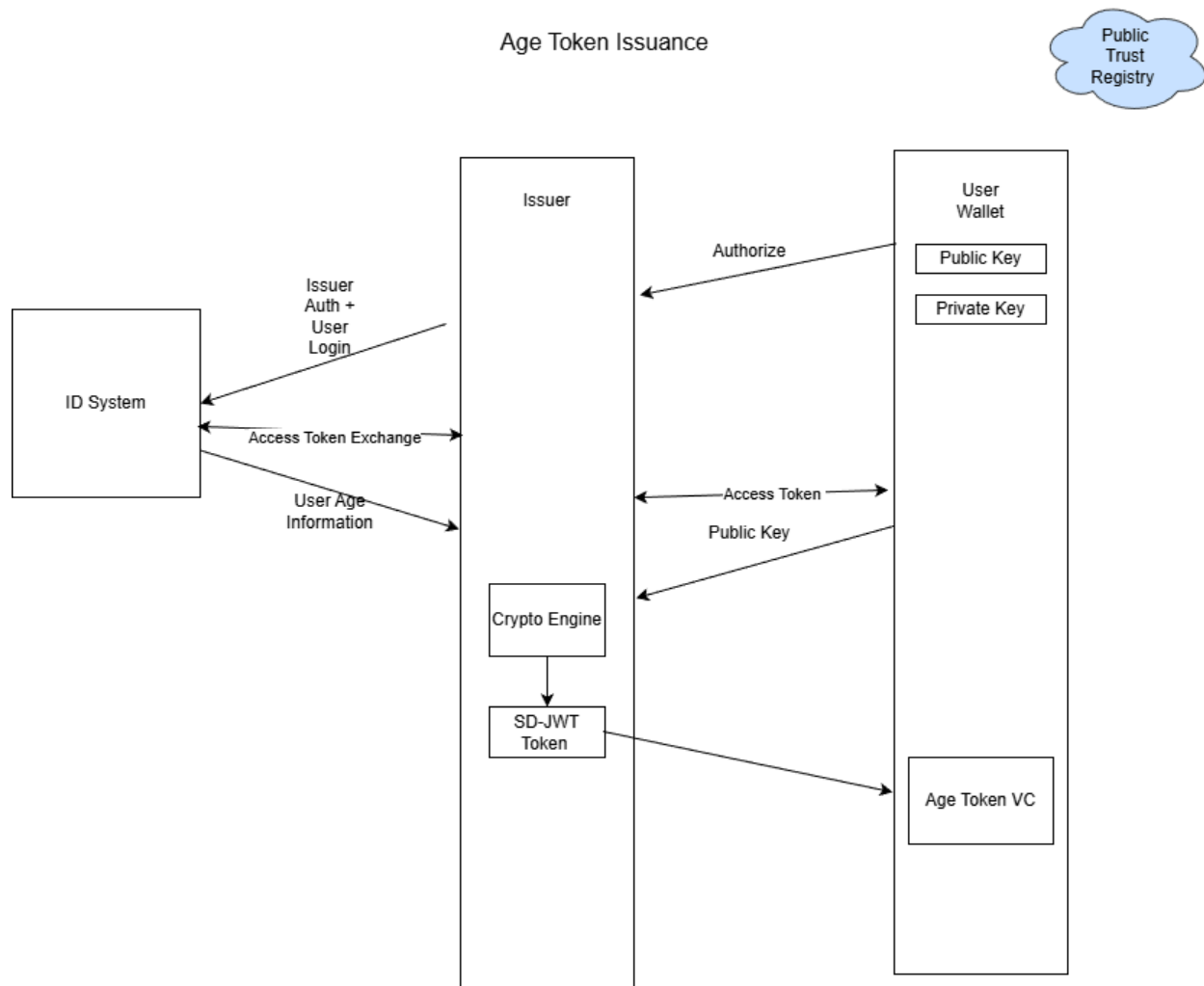


Child Safety Architecture



Age Token Issuance Flow



PHASE 0 — Wallet Initialization (Local Only)

Inside secure enclave:

0.1 Generate Wallet Key Pair

```
(wallet_private_key, wallet_public_key)
```

Private key never leaves device.

PHASE 1 — Wallet Starts Issuance (OIDC4VCI Layer)

1.1 Wallet Discovers Issuer Metadata

Wallet performs:

```
GET https://issuer/.well-known/openid-credential-issuer
```

Issuer responds with its API- endpoints

1.2 Wallet Starts OAuth Authorization Code Flow

Wallet opens browser to:

```
https://issuer/authorize?  
  client_id=wallet_app  
  response_type=code  
  scope=openid age_credential  
  redirect_uri=wallet://callbackstate=xyz
```

PHASE 2A — Authentication at ID Platform

2.1 Issuer Redirects to ID Platform

Issuer redirects browser to:

```
https://id.gov/authorize?  
  client_id=issuer
```

```
response_type=code
scope=openid profile
redirect_uri=https://issuer.gov/callback
```

2.2 User Logs In

User authenticates at ID Platform.

- Password + OTP
- Biometrics
- National ID verification

2.3 ID Platform Returns Auth Code to Issuer

```
https://issuer.gov/callback?code=AUTH_CODE
```

Issuer now exchanges code:

```
POST https://id.gov/token
```

Receives:

```
{"id_token":"...","access_token":"...","expires_in":3600}
```

2.4 Issuer Validates ID Token

Issuer verifies:

- Signature via ID Platform

Extracts:

```
{"sub":"opaque_id","dob":"2011-04-05","jurisdiction":"IN","assurance":"high"}
```

PHASE 2B- Age Estimation Pathway

STEP 1 — Wallet Submits Age Signals

Issuer prompts wallet:

| "Provide required signals for age estimation"

Can be

- Live selfie capture
- Short video
- Liveness detection
- Behavioral signals

Wallet sends encrypted signals to issuer.

Issuer forwards to Age Estimation Service

STEP 2 — Age Estimation Service Runs Model

The external service:

- Runs ML model
- Produces structured result:

```
{  
  "age_range": [12, 14],  
  "confidence": 0.93,  
  "model_id": "PAC_v3",  
  "liveness_verified": true, "timestamp": ...  
}
```

STEP 3 — Issuer Verifies Estimation Result

Issuer verifies:

- Service signature (if service signs response)
- Model version approved
- Confidence $\geq$ required threshold
- Liveness check passed

PHASE 3 — Age Determination

Issuer computes:

```
age_band
```

Build canonical claim set:

```
{
  "age_band": "13-15",
  "assurance_level": "high",
  "jurisdiction": "IN"
}
```

Determine expiry:

```
exp = next milestone birthday 0R90 days
```

PHASE 4 — Wallet Exchanges Authorization Code (OIDC4VCI)

Now wallet continues OIDC4VCI flow.

4.1 Wallet Receives Authorization Code

Browser redirects to:

```
wallet://callback?code=AUTH_CODE_FROM_ISSUER
```

Wallet now calls:

```
POST https://issuer.gov/token
```

Issuer returns:

```
{"access_token":"...", "c_nonce":"nonce123"}
```

This `c_nonce` is challenge for holder binding.

PHASE 5 — Holder Binding

Wallet sends credential request along with its public key:

```
POST https://issuer.gov/credential
Authorization: Bearer access_token
```

Body:

```
{"proof":{"proof_type":"jwt","jwt": sign(wallet_private_key,
c_nonce)}, "credential_definition":{"type":"AgeCredential"}}
```

Issuer binds credential to wallet key:

```
"cnf":{"jwk": wallet_public_key}
```

PHASE 6 — SD-JWT Construction

For each claim:

Issuer:

1. Generates random salt.
2. Create disclosure array:

```
["salt", "age_band", "13-15"]
```

3. Base64url encode.
4. Hash it (SHA-256).
5. Insert digest into `_sd` array.

Repeat for each selectively disclosable claim.

Final JWT payload:

```
{"_sd": [digest1, digest2, digest3],  
  "cnf": {"jwk": wallet_public_key},  
  "iss": "issuer.gov",  
  "iat": ...,  
  "exp": ...}
```

PHASE 7 — HSM Signing

Issuer sends JWT to HSM:

```
sign(private_key_kid_X, jwt)
```

Produces signature.

Issuer now has:

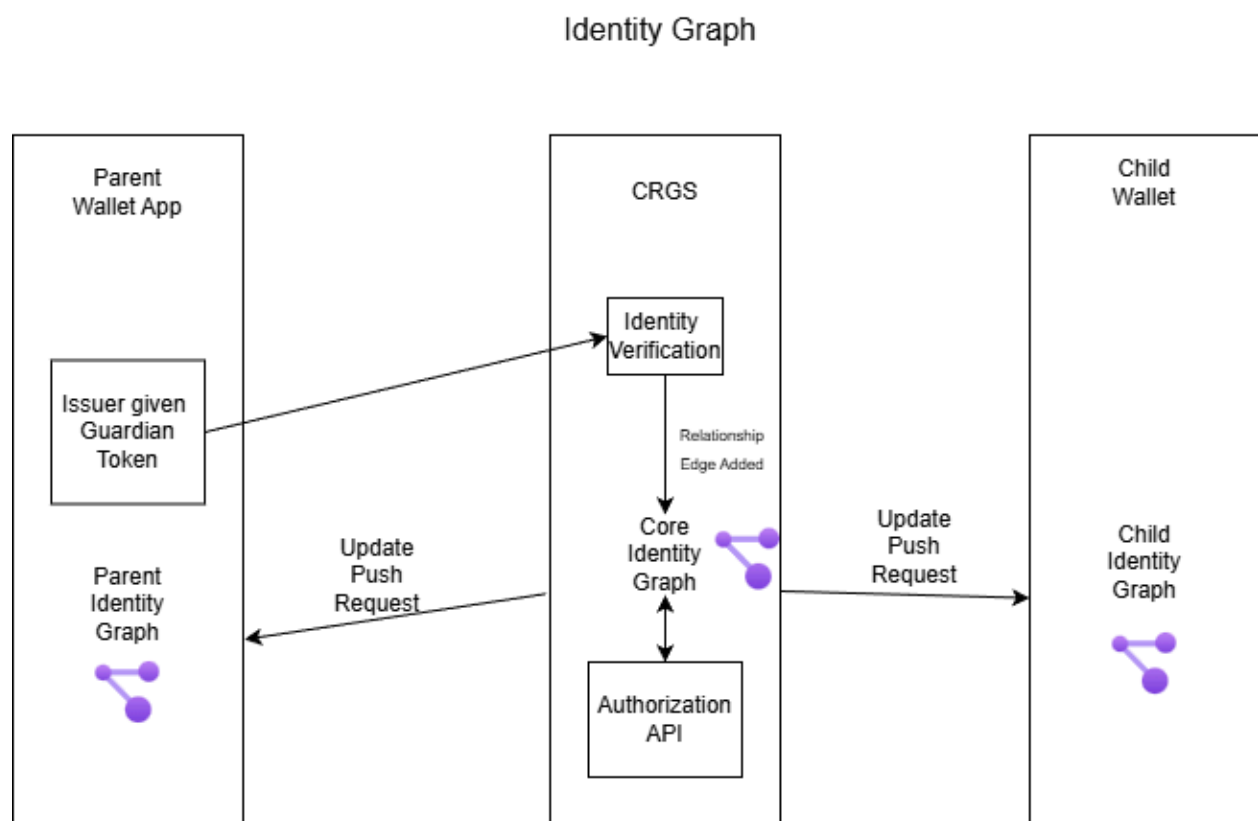
```
SD-JWT  
+ disclosures
```

PHASE 8 — Credential Delivery

Issuer returns:

```
{"credential":"SD-JWT","disclosures":[...]}
```

CRGS Flow



PHASE 1 — Guardian Identity Acquisition

Before CRGS is involved:

1.1 Guardian Gets Guardian Token (From Issuer)

Guardian wallet already holds:

- A Guardian Identity Token (SD-JWT or equivalent)
- Issued by trusted issuer
- Contains:
 - `sub`
 - `assurance_level`
 - `jurisdiction`
 - Possibly guardian eligibility claim

This proves guardian identity.

CRGS will trust this issuer.

PHASE 2 — Guardian Requests Relationship Addition

Guardian wants to link themselves to child.

2.1 Parent Wallet Initiates Link Request

Parent wallet sends to CRGS:

```
POST https://crgs.gov/link-request
Authorization: Bearer guardian_identity_token
```

Body:

```
{"child_reference": "child_identifier", "requested_role": "guardian-of"}
```

PHASE 3 — Relationship Edge Creation

Once legitimacy verified:

CRGS writes authoritative edge:

```
guardian_sub —(guardian-of, scope, capabilities)→ child_sub
```

Edge includes:

```
{"role": "guardian-of", "capabilities": ["policy.author", "policy.view", "policy.activate"], "scope": {"valid_from": "...", "valid_until": "...", "jurisdiction": "IN"}}
```

Graph update committed in CRGS database

PHASE 4 — Graph Snapshot Push

CRGS now updates wallets.

Two pushes occur:

4.1 Push to Guardian Wallet

```
POST https://guardian-wallet/update
```

Contains:

- Updated relationship proof
- Graph version number

Guardian wallet updates local identity graph snapshot.

4.2 Push to Child Wallet

```
POST https://child-wallet/update
```

Contains:

- Relationship proof token
- Version increment

Child wallet verifies:

- Signature via CRGS public key
- Version number > current

Then updates:

```
LocalIdentity GraphSnapshot
```

Stored securely.

PHASE 5 — Device-Side Storage

Both wallets now hold:

```
SignedGraphSnapshot
```

Contains:

- All relationship edges relevant to that identity
- Signed by CRGS
- Versioned

PHASE 6 — Runtime Authorization (Later Usage)

When policy engine needs authorization:

Wallet calls:

```
POST https://crgs.gov/authorize
```

Payload:

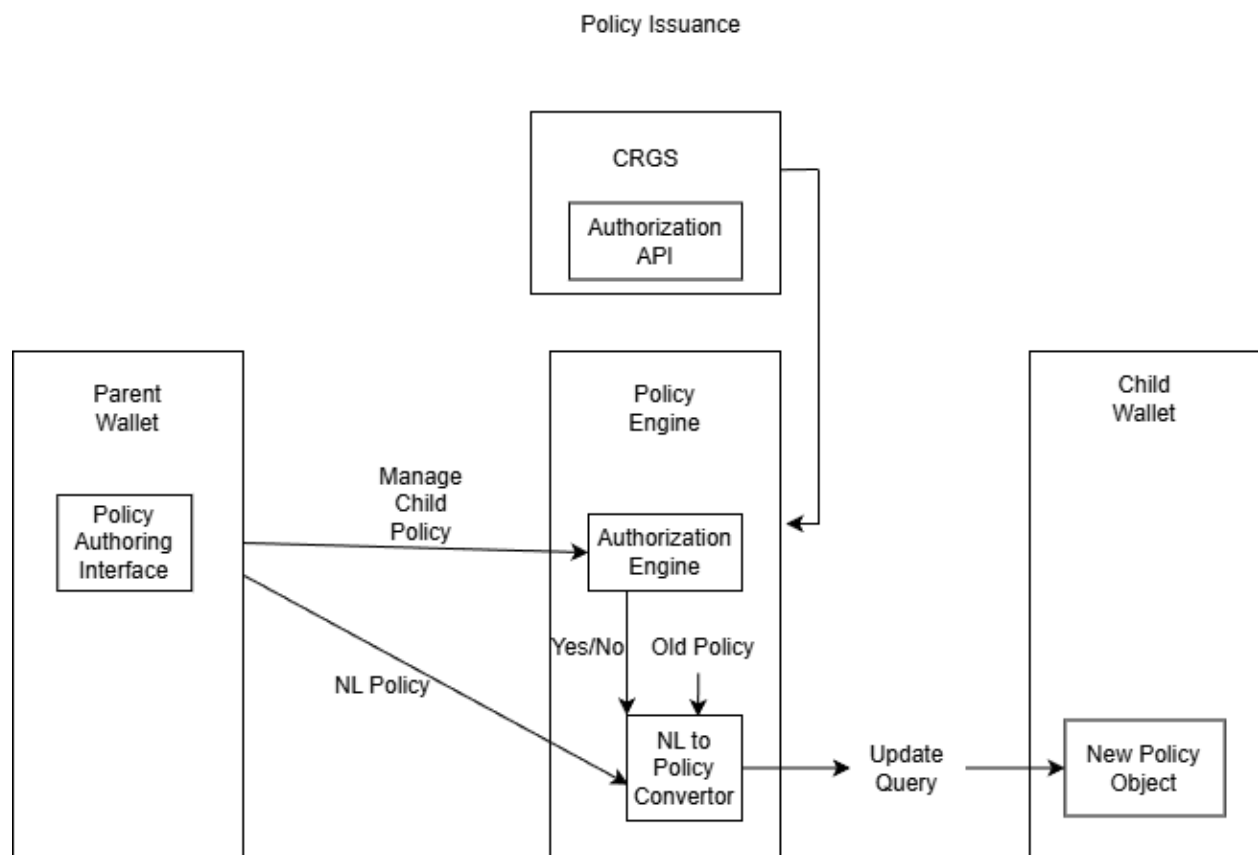
```
{"child_id":"...","actor_id":"...","requested_capability":"policy.author"}
```

CRGS verifies graph and returns:

```
{"authorized":true,"relationship_proof":"signed_token"}
```

Wallet verifies signature.

Policy Creation and Issuance



Phase 1 — Guardian Starts Policy Authoring

Step 1.1 — Guardian opens “Manage Child Policy”

Guardian wallet calls:

```
POST https://nlpps.gov/initiate-policy-session
```

NLPPS responds:

```
{"policy_session_id":"psess_123","nonce":"nonce_abc"}
```

Phase 2 — Authorization Check with CRGS

Before allowing policy creation:

NLPPS must verify:

| Is this guardian allowed to author policy for this child?

NLPPS sends to CRGS:

```
{"child_ref":"child_123","actor_ref":"guardian_789","requested_capability":"policy.author","context":{"operation":"create_policy"}}
```

CRGS performs:

1. Relationship lookup
2. Capability check
3. Scope validation
4. Delegation check

CRGS returns:

```
{"authorized":true,"relationship_proof":{"... signed ...}}
```

If unauthorized → NLPPS aborts.

Phase 3 — Guardian Submits Natural Language Policy

Guardian inputs:

- No religious content
- No usage after 10pm
- Block chat with unknown adults
- Allow educational content

NLPPS stores raw NL text temporarily.

Phase 4 — Policy Compilation (NL → Formal Rules)

NLPPS transforms NL into structured policy object from the policy rules set by multiple actors

Example:

```
{
  "policy_id": "pol_456",
  "child_id": "child_123",
  "rules": [
    {
      "effect": "deny",
      "capability": "content.religious"
    },
    {
      "effect": "deny",
      "capability": "interaction.chat.unknown_adult"
    },
    {
      "effect": "deny_after": "22:00"
    }
  ],
  "ed_on_age_band": "13-15",
  "version": 1,
  "status": "draft"
}
```

Phase 5 — Policy Activation

Guardian reviews and clicks “Activate”.

NLPPS:

1. Verifies authorization still valid (CRGS re-check optional)
 2. Increments version
 3. Marks policy as `active`
 4. Generates Effective Policy Snapshot
-

Phase 6 — Delivery to Child Wallet

Model A — Pull-Based (Preferred)

Child wallet periodically calls:

```
GET https://nlpps.gov/effective-policy?child_ref=child_123
```

Includes wallet proof.

NLPPS returns latest signed policy snapshot.

Wallet verifies:

- NLPPS signature
- Expiry
- Version monotonicity

Stores locally.

Model B — Push-Based

NLPPS sends:

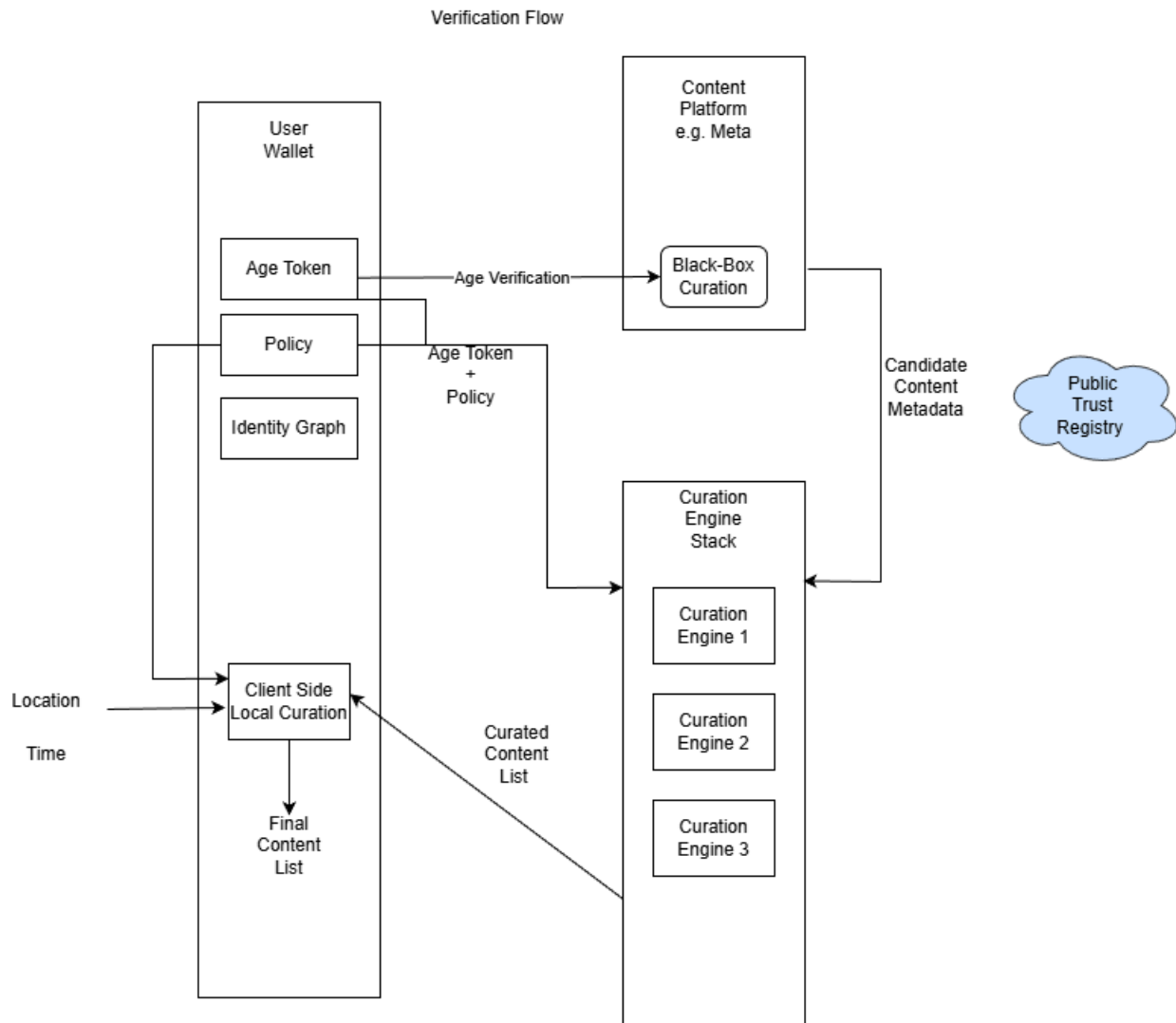
- Encrypted notification
 - Child wallet fetches snapshot
-

Phase 7 — Guardian Wallet Receives Confirmation

Guardian wallet receives:

- Policy ID
- Version
- Status
- Timestamp

Verification Flow



Step 1 — User Logs Into Platform

User sends content request to the platform.

Step 2 — Platform Creates OIDC4VP Request

Platform sends presentation request to wallet:

```

{"nonce": "random123",
 "audience": "platform.example",

```

```
"requested_credentials":["AgeCredential","PolicySnapshot"]
}
```

Step 3 — Wallet Prepares Presentation

Wallet prepares two objects:

4A — Age Presentation (SD-JWT Selective Disclosure)

Wallet reveals only required claims:

Example disclosures:

- age_band
- assurance_level

Wallet creates holder binding proof:

```
sign(wallet_private_key, nonce || audience)
```

This proves wallet owns `cnf` key.

4B — Policy Snapshot (Sent only to the 3rd party Curation Engines)

Wallet attaches:

```
Signed EffectivePolicySnapshot
```

It is already minimal and signed by NLPPS.

Step 5 — Wallet Sends Presentation

Wallet sends:

```
{
  "age_token":
  {"sd_jwt":"...", "disclosures":[...], "holder_proof":"..."},
  "policy_snapshot":{"jwt":"...signed by NLPPS..."}
}
```

Step 6 — Curation Platform Verification Phase

6A — Verify Age Token

Platform:

1. Fetches Issuer public key from trust registry.
2. Verifies SD-JWT signature.
3. Recomputes disclosure hashes.
4. Verifies holder PoP signature.
5. Checks expiry.

If any fail → reject.

6B — Verify Policy Snapshot (At 3rd party Curation Engines)

Platform:

1. Fetches NLPPS public key.
2. Verifies policy snapshot signature.
3. Checks expiry.
4. Checks policy version consistency.

If signature invalid → reject policy.

Step 7 — Eligibility Decision

Platform applies eligibility logic:

```
If age_band < minimum → denyElseif assurance < required → denyElse → proceedto curation
```

This is pure age logic.

Step 8 — Curation Using Age + Policy

Now platform has:

- Verified age attributes

8.1 Platform guardrails

Remove content violating:

- Legal age restrictions
- Jurisdiction rules

The resultant content metadata is returned to wallet which is transferred to 3rd party curation engine along with policy

8.2 Third-party curators

Each receives:

- content metadata
- policy object

Each removes content violating the policy

Step 10 — Wallet Final Enforcement

Wallet finally applies the following on the candidate list provided by the 3rd party curators:

- Applies time-based restrictions
- Applies last-mile overrides
- Shows explanations if needed

Wallet decision is final.